TECHNICAL DESIGN · CASE STUDY

Browser Agent for Automated Multi-Item Returns

Architecture, data flow and Excel schema for placing returns on Amazon and Flipkart, recorded per line item.

Browser automation agent | Amazon | Flipkart

6	8	10	5	5
core components	data-flow steps	columns per line item	failure types handled	open questions

The challenge in one line

A single order can hold several products, each with its own return window and its own return ID, and Amazon and Flipkart handle that differently. The agent has to detect the model, follow the correct path, and record an outcome for **every line item**, never just the order.

Note: figures on this page are counts of elements in the design (components, steps, columns, failure types, questions). The design contains no performance or volume data.

01 Objective and executive summary

Build a browser-based agent that reads pending return tasks from Excel, places the returns on the relevant platform on our behalf, and writes the outcome back per line item. It must work across platforms that treat multi-item orders differently and behave like a legitimate, well-mannered session.

Per-item by design

One Excel row is one SKU. Write-back, status and errors are always line-item level.

Failures stay local

One bad item never aborts its siblings. Nothing is silently dropped or left blank.

Adapters hide the mess

A common adapter interface keeps batch vs. sequential logic out of the write-back code.

Scope

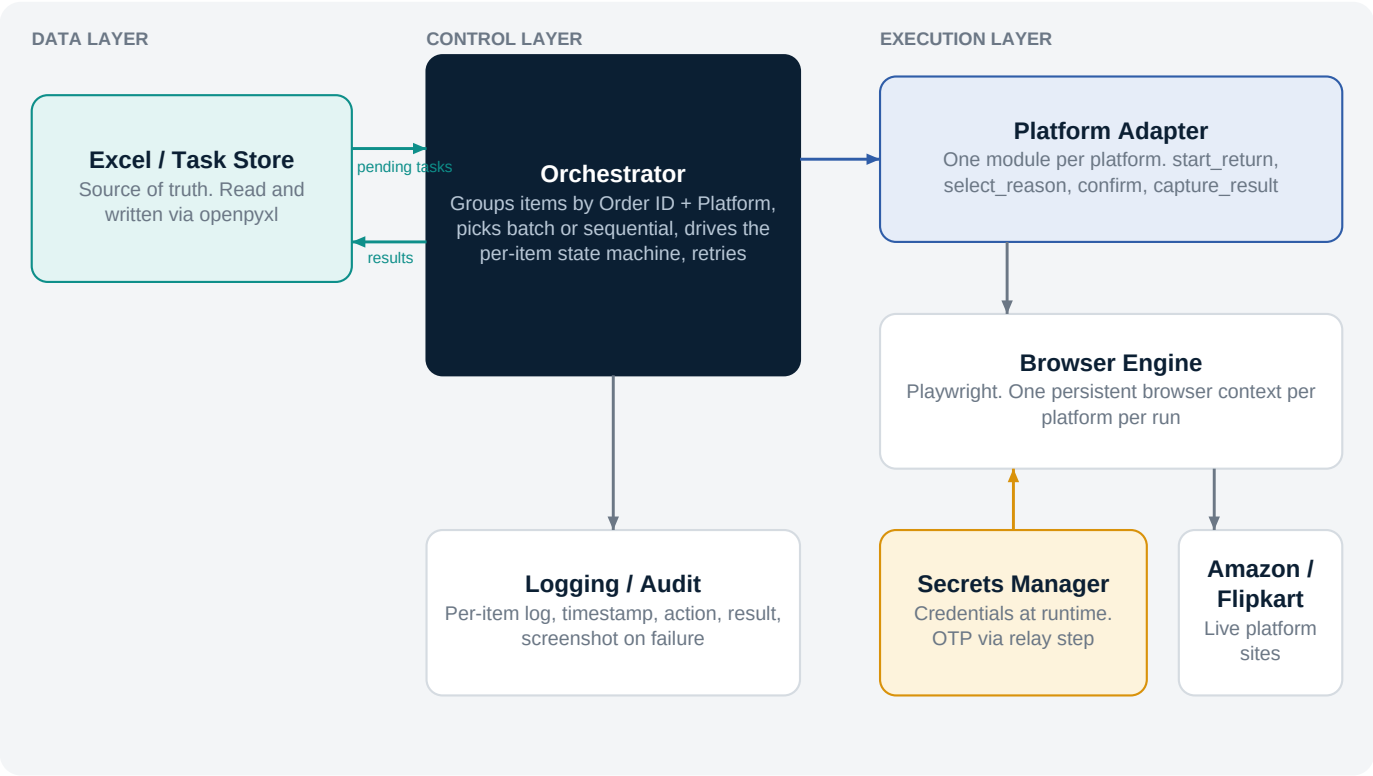
Automate placing returns on Amazon and Flipkart for pending line items tracked in Excel. Write the outcome (return ID, status, refund amount) back per line item and handle partial order success without human intervention, except where an item is flagged.

How the design answers the brief

Brief requirement	Where it is addressed	Design response
Read pending tasks from Excel	03 Data flow, step 1	Rows with Task status To Do / Pending, grouped by Platform + Order ID
Place the return per platform	03 and 05 Flow handling	Platform adapters with get_flow_type(); batch or sequential internally
Capture return ID, status, refund	03 Data flow, step 5	Scraped from the confirmation screen or order page
Write back per line item	04 Excel schema	10-column schema, one row per SKU, written after each item
Partial-success handling	06 Partial success	Independent items, eligibility pre-check, roll-up only at terminal states
Bot-detection avoidance (bonus)	08 Reliability	Human-like session hygiene; evasion techniques declared out of scope

02 Architecture

Six components with clear responsibilities. Excel feeds the Orchestrator, which calls the right Platform Adapter. The adapter drives the Browser Engine using credentials from the Secrets Manager. Results flow back through the Orchestrator to Excel and the audit log.



Component	Responsibility
Excel / Task Store	Source of truth for pending returns and final outcomes. Read and written through a library (openpyxl), never edited manually while the agent runs.
Orchestrator	Pulls pending tasks, groups line items by Order ID + Platform, decides batch vs. sequential flow, drives the per-item state machine and handles retries.
Platform Adapter	One module per platform implementing a common interface (start_return, select_reason, confirm, capture_result). Encapsulates platform-specific selectors and flows.
Browser Engine	Playwright. One persistent browser context per platform per run.
Secrets Manager	Stores and retrieves login credentials. OTP is handled through a manual or queued relay step, never stored in the Excel file or in code.
Logging / Audit	Structured per-line-item log (timestamp, action, result, screenshot on failure) for traceability and human review.

03 Data flow

Eight steps from task load to teardown. Blue steps set up the run, teal steps execute and record each line item, amber steps close the order and the session.

1 Load tasks

Read rows where Task status is To Do / Pending, grouped by Platform + Order ID.

2 Flow detection

Adapter reports batch or sequential. A property of platform and order type, not guessed per run.

3 Session start

Open a browser context for the platform, log in, navigate to the order.

4 Per-item execution

Check return window. If eligible: select item, reason, refund / pickup option, confirm. If not: skip and log Out of window.

5 Capture result

Scrape Return ID, status and refund amount from the confirmation screen or order page.

6 Write-back

Write to the item row right after it completes, so a crash never loses finished work.

7 Order roll-up

Order is Done only when every line item has a terminal state: Done, Out of window or Needs human review.

8 Teardown

Close the browser context and move to the next platform / order group.

Why write-back happens immediately
Results are written to each line item's row as soon as that item completes (step 6). If the run crashes mid-order, every item already finished keeps its recorded outcome.

Why eligibility is checked first
The return window is verified against the platform before any attempt (step 4). Eligibility is never inferred from a failure.

04 Excel schema

One row is one line item (SKU). There are no order-level-only rows.

Column	Type	Written by	Notes
Platform	text	Input	Amazon, Flipkart
Order ID	text	Input	Used to locate the order
Product / SKU	text	Input	Used to match the line item on the platform
Return window	date	Input / verified	Verified against the platform at run time
Return ID	text	Agent	Captured from the platform; blank if not placed
Return status	enum	Agent	Placed / Failed / Out of window
Refund amount	number	Agent	As shown by the platform
Task status	enum	Agent	To Do / Pending / Done / Needs human review
Timestamp	datetime	Agent	When the action ran
Error note	text	Agent	Populated only on Failed / Needs human review

Return status

Describes the platform outcome for that item: Placed, Failed or Out of window.

Task status

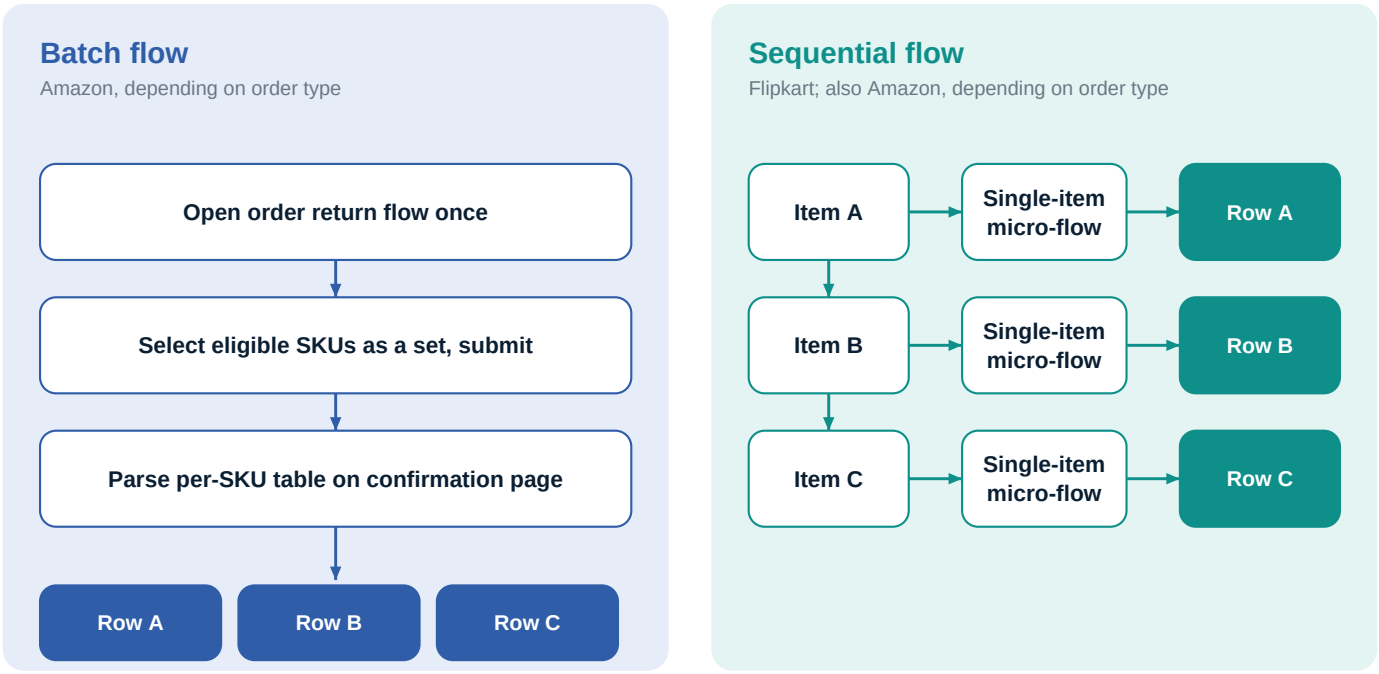
Describes whether the agent's job on that row is finished: To Do, Pending, Done or Needs human review.

How the two statuses pair
Out of window pairs with Task status **Done** because nothing more can be done. A scraping or selector failure leads to **Needs human review** regardless of Return status.

05 Batch vs. sequential flow

Both models can appear on Amazon depending on order type, while Flipkart is sequential. Rather than re-detecting per run, the adapter exposes one method:

```
adapter.get_flow_type(order_id) -> "batch" | "sequential"
```



Either way, the adapter returns exactly one result object per line item. The batch / sequential split stays inside the adapter.

06 Partial-success handling

Illustration based on the example in the brief: A and B are placed even though C is past its return window.

LINE ITEM	ELIGIBILITY CHECK	AGENT ACTION	RETURN STATUS	TASK STATUS
Item A	In window	Return placed	Placed	Done
Item B	In window	Return placed	Placed	Done
Item C	Past window	Skipped, logged, no retry	Out of window	Done

Rules that govern partial success

- Line items are processed independently. One failure never aborts sibling items in the same order.
- Eligibility (return window) is checked **before** attempting a return, not inferred from a failure.
- Any item that cannot be completed (ineligible, selector break, platform error, ambiguous match) is recorded with an error note, never silently dropped or left blank.
- Order-level Done is a derived roll-up, computed after all line items reach a terminal state. The agent does not set it directly.

Reviewer note: one point to align on
The brief says skipped or ineligible items should be flagged for human review. The design records **Out of window** as a terminal **Done** with a log entry, and reserves **Needs human review** for items that cannot be resolved (selector break, platform error, ambiguous match). Confirm this mapping is what the business wants.

07 Credentials and OTP handling

No credentials or phone numbers live in the Excel file, the codebase or the logs. Login credentials are pulled from a secrets manager (environment-injected secret or vault) at runtime. OTP login cannot be fully unattended by design, because the OTP is sent live to a phone.

Pattern A: manual relay

Agent pauses at the OTP prompt and a human enters the code once per session. Long-lived sessions via saved browser storage state minimise how often this is needed.

Pattern B: SMS-forwarding webhook

If the business phone line can forward OTP SMS to an API endpoint, the agent polls or waits for it. Needs an explicit integration, not assumed by default.

Session reuse
Playwright's storageState (cookies + local storage) is persisted after a successful login, so later runs reuse the session instead of re-triggering OTP every time.

08 Reliability and human-like behaviour

For the agent's own legitimate account automation, acting like a real session means good general automation hygiene, not defeating platform anti-fraud systems.

In the design

- Randomised, human-scale delays between actions, not a fixed machine-speed rhythm
- Real user-agent and viewport; one action at a time, no parallel hammering of the same account
- Normal page-load waits (Playwright auto-wait) instead of blind sleeps
- Reuse a persistent, logged-in session rather than a fresh identity per return
- Rate-limit to a sensible number of returns per session / day

Out of scope

- Fingerprint spoofing
- Proxy rotation to evade IP-based detection
- Defeating CAPTCHA or anti-bot challenges

These risk violating platform Terms of Service regardless of the business's intent, so this design does not cover them.

09 Error handling and retry

Failure type	Behaviour
Selector not found / page layout changed	Retry once after short backoff. On repeat failure, mark Needs human review and attach a screenshot to the log.
Network / timeout	Retry up to 2x with backoff, then Needs human review.
Return window expired	No retry. Immediately Out of window with Task status = Done.
Platform rejects reason / refund option	Log the platform message verbatim in Error note and mark Needs human review.
Login / session failure	Halt the platform run and alert the operator. Do not silently skip all remaining tasks for that platform.

10 Open questions

To confirm before build.

1. **OTP relay**
   Is OTP relay manual (a human types the code) or is an SMS-forwarding integration available?

2. **Excel location and format**
   Local file, SharePoint or Google Sheets? This affects the read/write library and locking during concurrent runs.

3. **Run frequency and volume**
   A few returns a day or hundreds? This decides whether session reuse or fresh logins per run makes sense.

4. **Headless or headful**
   Headful is easier to debug and for manual OTP entry.

5. **Currency**
   Multi-currency or single-currency orders, for refund amount parsing.

Conclusion

What this design delivers
A per-line-item return agent that detects batch or sequential flows behind one adapter interface, completes every eligible item even when siblings fail, records each outcome immediately, keeps credentials out of files and code, and stays within normal, platform-respectful automation behaviour. Five open questions remain before build.